Class Test:-

M. Pranay
192525382
CSA0405
Operating system.

1) Evolution of Operating systems University Computer upgrade :-

44/50

1) Batch Processing System:-

→ Executes a group of Jobs one after another without user interaction.
→ CPU handles one job at a time.
→ CPU utilization is low because it waits during I/O Operations.
→ Response time is high.

Example:- Salary Processing.

2. Multiprogramming System:-

→ Multiple Programs are stored in memory simultaneously.
→ When one Program waits for I/O, the CPU executes another Program.
→ Reduces CPU idle time
→ Improves CPU utilization

Ex:- Running multiple Programs in a Computer.

3. Time-sharing systems:-

→ Allows multiple users to use the computer simultaneously.
→ CPU time is divided into small time slots.
→ Provides quick responses to users.

Ex:- Multiple students accessing a university server.

4. Modern Operating systems:-

→ Supports multitasking and multiprocessing.
→ Provides security and memory management.
→ Allows multiple applications to run simultaneously.
→ Offers high CPU utilization and faster response time.

Ex:- Windows, Linux and macOS.

Comparision:-

System	CPU utilization	Response time
Batch	low	High
multiprogramming	Improved	Moderate
Time-sharing	High	low
Modern OS	Very High	Very low

Conclusion:- Operating system evolved from batch processing to modern systems.

---

2) Desktop systems vs Multiprocessor systems – Symmetric and Asymmetric multiprocessing.

1) Desktop system:-

→ Usually has a single CPU.
→ Supports one user and multitasking.
→ Uses time-sharing to execute tasks.
→ Suitable for personal and office work.

Multiprocessor System:-

→ Contains two or more CPUs sharing memory and I/O devices.
→ All processors work under one operating system.
→ Supports parallel processing.
→ Provides better performance and reliability.

Comparision:-

System	CPU utilization	Response time
Batch	low	High
multiprogramming	improved	Moderate
Time-sharing	High	low
Modern OS	very High	very low

Conclusion:- Operating system evolved from batch processing to moder systems.

---

2) Desktop systems vs Multiprocessor systems – symmetric and Asymmetric multiprocessing.

1) Desktop system:-

→ Usually has a single CPU.
→ Supports one user and multitasking.
→ Uses time-sharing to execute tasks.
→ Suitable for personal and office work.

Multiprocessor System:-

→ Contains two or more CPUs sharing memory and I/O devices.
→ All Processors work unde one Operating system.
→ Supports Parallel Processing.
→ Provides better Performance and reliability.

2. Asymmetric multiProcessing (AMP):-

→ one Processor acts as the master, and others acts as slaves
→ master Processor controls the OS, scheduling and I/O operations.
→ slave Processors execute the tasks assigned by the master.
→ It has a master - slave structure

3. Symmetric multiProcessing (SMP):-

→ All Processors have equal responsibilites.
→ Each Processor can execute os tasks and user Programs.
→ Tasks are shared among Processors.
→ It improves Performance through Parallel execution.

Ex:- Linux and windows server support SMP.

4) Example of Task scheduling.

Consider six tasks, each requiring 4 CPUs time units.

→ Single Processor:- one CPU executes all six tasks, requiring 24 time units.
→ 4-CPU SMP system:- Four CPUs shares the tasks and complete them in 8 time units.

Scheduling features:-

→ load balancing distributes tasks equally.
→ Processor affinity keeps tasks on a Preferred CPU.
→ Locks Prevent conflicts when sharing resources.

Comparision of SMP and AMP:-

SMP:-

1) All CPUs are equal.
2) Any CPUs can run OS tasks.
3) Better load balancing.
4) More flexible and scalable.
5) More complex os design.

AMP:-

1) Master - slave structure.
2) Master controls OS tasks
3) Master may become a bottleneck.
4) Less flexible.
5) Simpler coordination.

Conclusion:- MultiProcessor System Provide better Performance than single Processor desktops.

3) System Calls - File copying Program.

a)

1) Sequence of system calls.

When copying data from a source file to a destination file, the following steps are performed

1) open():- Open the source file in read mode
2) create():- creates the destination file in write mode.
3) read():- Reads data from the source file.
4) write():- writes the data into the destination file.
5) Loop:- Repeats until the end of the file.
6) close():- closes both files.
7. exit():- Terminates the Program.

2) Types of system calls:-

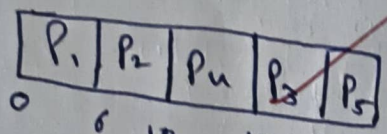
Types	Example:-
Process control	fork(), exec(), exit()
File Management	open(), read(), write(), close()
Device management	ioctl(), read(), write()
Information maintenance	getpid(), time(), data()
communication	Pipe(), shmget(), send(), receive()
Protection	chmod(), chown().

Conclusion:- System Calls Provides communication b/w User Programs and the OS kernel.

b) Non-Preemptive (FCFS):-

Process ID	AT	BT	CT	TAT	WT	Priority
P_1	0	6	6	6	0	3
P_2	1	4	10	9	5	1
P_3	2	5	17	15	10	4
P_4	3	2	12	9	7	2
P_5	4	3	20	16	13	5
				58	34	

Gantt chart:-



Average waiting time = $\frac{34}{5} = 7.6$

Average Turnaround time = $\frac{58}{5} = 11.6$.

4) Deadlock Detection Algorithm:-

Given:-

Resource A = 7 instances
Resource B = 2 instances
Resource C = 6 instances.

Process	Allocation (A B C)	Request (A B C)	work after completion (A B C)
P_0	0 1 0	0 0 0	0 1 0
P_1	2 0 0	2 0 2	3 1 3
P_2	3 0 3	0 0 0	5 2 4
P_3	2 1 1	1 0 0	5 2 6
P_4	0 0 2	0 0 2	7 2 6

a) Available vector:-

A = 0+2+3+2+0 = 7
B = 1+0+0+1+0 = 2
C = 0+0+3+1+2 = 6

Available vector = (Total − Allocation)
= (7,2,6) − (7,2,6)
= (0,0,0)

b) Deadlock Detection Algorithm:-

$P_0 - P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

Conclusion:- All processes complete successfully.
Hence, the system is not deadlocked.

c) Additional Request by $P_2 = (0,0,1)$:-

Process	Request	work	Result:-
P_1	2 0 2	0 1 0	Cannot execute
P_2	0 0 1	0 1 0	Cannot execute
P_3	1 0 0	0 1 0	Cannot execute
P_4	0 0 2	0 1 0	Cannot execute.

---

5) a) Producer – Consumer Problem:-

Given:-

Initial Counter = 10
Producer Performs Counter++
Consumer Performs Counter--

Time	Process	Operation
T_0	Producer	10
T_1	Producer	11
T_2	Consumer	10
T_3	Consumer	9
T_4	Producer	11
T_5	Consumer	9

Final Counter value = 9

Posible value = 10, because one increment and one decrment cabel each other.

c)

b) Readers - writers Problem:-

Initial values:-

mutex = 1

wrt = 1

readcount = 0

Execution	Readcount	Status
R_1 arrives	1	Reading
R_2 arrives	2	Reading
W_1 arrives	2	Blocked
R_3 arrives	3	Reading
R_1 leaves	2	Reading
R_2 leaves	1	Reading
R_3 leaves	0	write gets access.

Blocked Process:- W_1 is blocked until all readers finish.

Starvation Risk:- Writer starvation may occur because continuous arrival of readers can prevent the writer from accessing the resource.

C) Dining Philosophers Problem :-

1. Given,
   - → 5 Philosophers
   - → 5 Chopsticks
   - → Each philosopher needs 2 Chopsticks.

1) Maximum Philosophers eating simultaneously :-

$$[5/2] = 2$$

Maximum = 2 Philosophers.

2) How deadlock occurs :-

→ Every philosopher picks up the left chopstick.
→ Each waits for the right chopstick.
→ Nobody releases their chopstick
→ This causes Deadlock.

3) Solution :- limit the table to 4 Philosophers at a time. This ensures at least one philosopher can obtain both chopsticks and eat, preventing deadlock.